SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I __R Lakshmi Narasimha(192472344)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital Doctor Shift Scheduling Using Backtracking Search

Problem: Assign three doctors (D1, D2, D3) to Morning, Afternoon, and Night shifts while satisfying all constraints.

Constraints

D1 cannot work the Night shift.

D2 must work before D3.

Only one doctor per shift.

D3 cannot work the Morning shift.

Every doctor must receive exactly one shift.

Step-by-step backtracking

Step	Assignment	Constraint check	Result
1	D1 → Morning	D1 is not assigned Night	Valid
2	D2 → Afternoon	D2 works before D3	Continue
3	D3 → Night	D3 is not assigned Morning	Valid
4	Check all shifts	One doctor per shift; all assigned	Success

The ordering constraint is satisfied because D2's shift occurs before D3's shift.

Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Result: Backtracking Search finds a valid assignment satisfying all constraints without needing to backtrack from this solution.

2. Robot Navigation in a 5×5 Grid Using BFS

Problem: A robot must move from Start (S) to Goal (G), avoiding obstacles. Each move costs 1, and movement is allowed only Up, Down, Left, and Right.

Algorithm

Breadth-First Search (BFS) explores nodes level by level. Since every move has equal cost, BFS finds a shortest path in terms of the number of moves.

Note: The assessment text does not specify the positions of S, G, and the obstacles in the 5×5 grid. Therefore, the exact node expansion sequence and final path cannot be determined uniquely from the provided information.

Step-by-step procedure

Place the start node S in the queue.

Remove the front node from the queue and mark it as visited

Check its neighbours in the order Up, Down, Left, Right.

Ignore cells that are outside the grid, blocked, or already visited.

Add valid neighbours to the queue and record their parent nodes.

Repeat until the goal G is reached.

Trace the parent nodes backward from G to S to obtain the shortest path.

Pseudocode

```
BFS(S, G):
   Queue = [S]
   Visited = {S}

   while Queue is not empty:
      Current = remove_front(Queue)

      if Current == G:
         return reconstruct_path(G)

      for each neighbour of Current:
         if neighbour is valid and not visited:
            Visited.add(neighbour)
            Parent[neighbour] = Current
            add_to_back(Queue, neighbour)

   return "No path found"
```

Priority queue and heuristic clarification

The question mentions Manhattan Distance, defined as

$$ h(n)=|x_n-x_G|+|y_n-y_G| $$

However, standard BFS uses a FIFO queue and does not use a heuristic. Manhattan Distance is typically used in Greedy Best-First Search or A* Search. If the assessment requires BFS specifically, use the FIFO queue for node expansion and calculate the heuristic separately if requested.

The path cost is:

$$\text{Total cost}=\text{Number of moves}\times 1$$

Result: BFS finds an optimal shortest path if one exists. The exact path and cost require the missing grid layout.

3. Autonomous Rescue Robot in a Disaster-Affected Building

a) Analyze the constraints and their effects

Constraint	Effect on decision-making
Movement in four directions	The robot can move Up, Down, Left, or Right.
Obstacles	The robot must avoid blocked cells and find an alternative route.
Limited sensing	It can observe only adjacent cells, so it cannot know the entire environment initially.
Movement cost = 1	Every normal move adds 1 to the total cost.
Risky-zone cost = 2 extra	Entering a risky zone costs $1+2=3$.
Minimize total cost	The robot should select the least-cost safe route to the survivor.
Dynamic knowledge	The robot must update its map as it discovers new obstacles and paths.

b) Solution using Uniform Cost Search and an Online Search Agent

Chosen approach: An Online Search Agent combined with Uniform Cost Search (UCS).

UCS expands the node with the lowest accumulated path cost first. The online component allows the robot to discover the environment gradually and update its knowledge.

Steps involved

Start at position S and initialize the path cost to zero.

Observe the four adjacent cells.

Identify obstacles, safe cells, and risky zones.

Assign a cost of 1 to normal moves and 3 to moves entering risky zones.

Add discovered, reachable cells to a priority queue using their cumulative path costs.

Expand the cell with the lowest cost and update the robot's map.

If a new obstacle is discovered, revise the planned route.

Continue until the survivor at G is reached.

Pseudocode

```
BEGIN
  Set current position = S
  Set cost(S) = 0
  Initialize priority queue with S

  WHILE survivor not reached
    Observe adjacent cells
    Update map and identify obstacles

    FOR each safe, reachable neighbour
      Move cost = 1

      IF neighbour is a risky zone
        Move cost = 3
      END IF

      Calculate new cumulative cost

      IF new cost is lower than known cost
        Update cost and parent
        Insert or update neighbour
        in priority queue
      END IF
    END FOR

    Expand the reachable cell
    with the lowest cumulative cost
  END WHILE

  Return the route and total cost
END
```

Result: The robot progressively discovers its surroundings and uses cost-based search to find a low-cost route to the survivor while avoiding known obstacles.

Note: Because no specific building grid or obstacle positions are supplied in the assessment text, a numerical route and total cost cannot be calculated.

c) Application of AI concepts

1. Intelligent agent

An intelligent agent perceives its environment through sensors and acts using its available information. The rescue robot observes adjacent cells, detects obstacles, plans movement, and navigates toward the survivor.

2. Rationality

A rational agent selects the action expected to maximize its performance based on the information available.

The robot acts rationally by:

Avoiding known obstacles.

Preferring lower-cost routes.

Considering the additional cost of risky zones.\Updating its decisions when new information becomes available.

Prioritizing safe navigation and successful rescue

3. Environment type

Property	Environment characteristic
Observability	Partially observable
Determinism	Stochastic or uncertain
Structure	Sequential
Changes	Dynamic
State representation	Discrete grid
Number of agents	Single-agent

The environment is partially observable because the robot can sense only adjacent cells. It is sequential because every movement affects subsequent decisions. It is dynamic because newly discovered obstacles or changing hazards can require route replanning.

d) Justification of the chosen approach

The combination of an Online Search Agent and UCS is suitable because:

Handles incomplete information: The online agent explores and updates its knowledge gradually.

Finds low-cost routes: UCS considers cumulative movement costs instead of only the number of steps.

Considers risky areas: Additional costs discourage unnecessary entry into dangerous zones.

Adapts to discoveries: The robot can update its map and replan when it encounters previously unknown obstacles.

Supports rational decisions: It selects actions based on the current map, path costs, and safety constraints

Conclusion: An Online Search Agent with UCS is an appropriate approach for rescue navigation under partial observability and variable movement costs. However, guaranteed global optimality requires sufficient map discovery and appropriate replanning; the robot cannot guarantee a globally optimal route from incomplete information alone.